

Detecting robotic grasping points using Keypoint R-CNN

[Start Assignment](#)

- Due Sunday by 23:59
- Points 10
- Submitting a file upload
- File types zip

IMPORTANT:

Each group consists of 3-4 students, and ideally at least one of you should have a CUDA-supported GPU on their computer.

Detecting robotic grasping points using Keypoint R-CNN

Introduction

Robotic harvesting requires accurate estimation of grasping points, even when crops are partially hidden under leaf occlusions. Traditional object detection networks such as Faster R-CNN or Mask R-CNN, when applied only to visible regions, produce offsets in the estimated grasping point, limiting their practical use in robotics.

The **Keypoint R-CNN** architecture introduces a keypoint detection branch, enabling end-to-end prediction of grasping points directly from images. This approach can improve precision and robustness under complex crop occlusions.

The goal of this assignment is to implement Keypoint R-CNN on a custom broccoli dataset, adapting both the architecture and dataloader for end-to-end keypoint detection:

Original image with leaf occlusion



Mask R-CNN



Keypoint R-CNN



Objective

Your task is to **build and train** a Keypoint R-CNN model (or similar model for keypoint detection) to predict the robotic grasping point on broccoli crops, using a custom image dataset. The dataset ([broccoli.zip](https://tilburguniversity.instructure.com/courses/21388/files/4159149?wrap=1) (<https://tilburguniversity.instructure.com/courses/21388/files/4159149?wrap=1>) [↓](#) (https://tilburguniversity.instructure.com/courses/21388/files/4159149/download?download_frd=1)) contains 1613 images and 1613 corresponding annotations in LabelMe (.json) format, where each annotation defines the circular region describing the shape of the broccoli head (center point and radius).

The objective is to adapt the Mask R-CNN architecture from the [torchvision_tutorial.ipynb](https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1) (<https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1>) [↓](#) (https://tilburguniversity.instructure.com/courses/21388/files/4352379/download?download_frd=1) and modify it for keypoint detection on the broccoli image dataset. Feel free to explore or implement different implementations of Keypoint R-CNN or different algorithms.

Procedure

1. Set up your environment and install pycocotools, and labelme

```
conda activate dl
```

```
pip install pycocotools
```

```
pip install labelme
```

2. Download the vision_utils from Canvas

From Canvas, download [vision_utils.zip](#)

(<https://tilburguniversity.instructure.com/courses/21388/files/4352378?wrap=1>)_ ↓

(https://tilburguniversity.instructure.com/courses/21388/files/4352378/download?download_frd=1)

and unzip it inside your local `dl` folder.

This will create:

```
dl/
├── vision_utils/
│   ├── __init__.py
│   ├── coco_eval.py
│   ├── coco_utils.py
│   ├── engine.py
│   ├── transforms.py
│   └── utils.py
```

3. Torchvision tutorial

Download the tutorial notebook [torchvision_tutorial.ipynb](#)

(<https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1>)_ ↓

(https://tilburguniversity.instructure.com/courses/21388/files/4352379/download?download_frd=1)

from Canvas.

Run and study all cells carefully to understand the code for finetuning Mask R-CNN on the Penn-Fudan Pedestrian Dataset.

Guidelines for implementing Keypoint R-CNN with custom dataloader

1. Explore and prepare the dataset

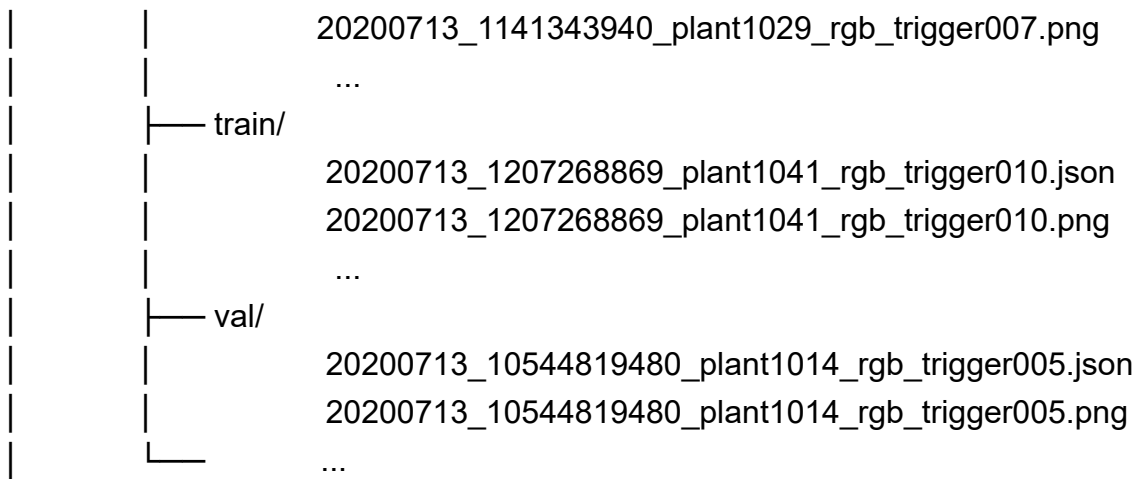
- Download and unzip [broccoli.zip](#)

(<https://tilburguniversity.instructure.com/courses/21388/files/4159149?wrap=1>)_ ↓

(https://tilburguniversity.instructure.com/courses/21388/files/4159149/download?download_frd=1) from Canvas.

- Inspect the images, the annotation structure (LabelMe `.json` files), and the dataset splits:

```
dl/
├── data/
│   └── broccoli/
│       ├── test/
│       └── 20200713_1141343940_plant1029_rgb_trigger007.json
```



2. Custom Dataloader

- Implement a custom dataloader capable of processing LabelMe-format JSON annotations.
- Modify the data target in the custom dataloader to match the Keypoint R-CNN requirements:
 - `keypoints` should be a float `torch.Tensor` of shape `[N, K, 3]`, where
 - **N** = number of objects
 - **K** = number of keypoints
 - **keypoint format:** `[x, y, v]`, with
 - `x`: center point x-coordinate in pixels
 - `y`: center point y-coordinate in pixels
 - `v`: visibility (for this task, set `v = 2`). Refer to [keypoint visibility](https://github.com/pytorch/vision/issues/5872) (<https://github.com/pytorch/vision/issues/5872>).

3. Model Architecture

- Modify the Mask R-CNN architecture from the [torchvision_tutorial.ipynb](https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1) (<https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1>) [↓](#) (https://tilburguniversity.instructure.com/courses/21388/files/4352379/download?download_frd=1) to a Keypoint R-CNN architecture. Or implement another algorithm.
- Find inspiration from [keypoint_rcnn.py](https://github.com/pytorch/vision/blob/main/torchvision/models/detection/keypoint_rcnn.py) (https://github.com/pytorch/vision/blob/main/torchvision/models/detection/keypoint_rcnn.py).

4. Training and Evaluation

- Train the model using the prepared broccoli dataset.

- Experiment with different hyperparameters to observe their effect on the detection performance.
- Monitor loss curves and visualize keypoint predictions.
- Evaluate the performance qualitatively (visual results) and quantitatively (coco_evaluator).

5. Debugging

- Use **Visual Studio Code** (<https://code.visualstudio.com/>) or another code editor instead of a Jupyter Notebook.
- Ensure your dataset paths, annotations, and image transforms are consistent.
- Verify your **keypoint tensor format** before model training.
- Debugging is challenging when the device is set to cuda! So please start training with:
`device = torch.device('cpu')`
- If the training and evaluation scripts don't crash, please continue training your model with:
`device = torch.device('cuda')`

Submission Files

- Python script (.py) or Jupyter Notebook (.ipynb).
- Please include the neural network weights in **.pt** or **.pth** format so that I can reproduce your results.
- Submit a short report summarizing the approach, key innovations, experiments you conducted, current challenges, potential improvements, and the final results. Please summarize the results with a few visual examples and proper evaluation metrics that were presented during the lecture.
- Make one ZIP file that contains the files above. Only ZIP files can be uploaded in the Canvas environment.

Method for Submission

Submit the ZIP file on Canvas (one submission per group). Use a distinct name:

DL2026_GA02_[GroupName].zip

Submission Deadline

The submission deadline is **Sunday May 24th 23.59 (11.59 pm)**.

Grading

The final grade is based on the following components:

- **Code (50%)**
- **Report (50%)**

Code (50%):

1. Data loading and preprocessing

- Correct implementation of a custom dataloader for LabelMe-format JSON annotations
- Proper extraction of bounding boxes, and keypoints from the annotation files
- Keypoint tensors correctly formatted as $[N, K, 3]$ with $[x, y, v]$ per keypoint, and visibility flag $v = 2$
- Use of data augmentation techniques (e.g., random flipping, color jitter, scaling) that are consistent across image and keypoint coordinates

2. Model approach and methodology

- Correct adaptation of the Mask R-CNN architecture to Keypoint R-CNN, or a well-motivated alternative approach to keypoint detection
- Appropriate configuration of the keypoint head (number of keypoints, keypoint coordinates)
- Hyperparameter tuning: learning rate, learning rate schedulers, batch size, number of epochs
- Appropriate choice of confidence threshold and Non-Maximum Suppression (NMS) settings

3. Training and evaluation

- Successful training on the broccoli dataset with loss curves monitored and reported
- Use of the COCO evaluator (`coco_evaluator`) for quantitative evaluation
- Use of appropriate evaluation metrics, such as OKS-based AP (Object Keypoint Similarity), and keypoint localization error
- Qualitative evaluation through visual inspection of predicted keypoints overlaid on image

4. Code quality and reproducibility

- Reproducibility ensured (e.g., random seeds set)
- Clean, well-structured, and properly commented code
- Model weights saved in `.pt` or `.pth` format so results can be reproduced
- An inference mode that is able to reproduce keypoint predictions using your model and optimized weights

Report (50%):

In the written report (maximum 10 pages), include the following:

- Summary of your approach
- A clear description of the implemented architecture and any modifications made to the baseline Mask R-CNN
- Justification of design decisions, such as:

Keypoint detection head: why it is suitable for predicting robotic grasping points on partially occluded crops

Data augmentation strategies chosen and how they account for the specific challenges of this dataset (e.g., leaf occlusions, lighting variation)

Choice of evaluation metrics and thresholds — do they favor precision, recall or keypoint localization, and which is most appropriate for the robotic harvesting use case?

- Reflection on results

Analysis of loss curves and quantitative evaluation metrics (e.g., OKS-AP, keypoint localization error)

Discussion of any trade-offs or difficulties encountered

A few visual examples showing predicted grasping points on test images, including both successful and failure cases

- Future improvements

Suggestions for improving model performance, such as alternative frameworks, architectures, algorithms, synthetic data augmentation, etc. References to relevant scientific papers or GitHub repositories where appropriate

References and useful links

- **Torchvision tutorial** (https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html) (original, please use the one that is provided on **Canvas** (<https://tilburguniversity.instructure.com/courses/21388/files/4352379?wrap=1>)_ ↓ (https://tilburguniversity.instructure.com/courses/21388/files/4352379/download?download_frd=1))
- **LabelMe installation and use** (http://youtube.com/watch?v=PtUO_H3DEc8) (from minute 13 it explains the circle annotation procedure)
- **Broccoli dataset** (<https://doi.org/10.4121/13603787.v3>)

Resit options

Resit options will be made accordingly, in the situation the group fails to submit on time or when the group receives a grade lower than 5.5.